

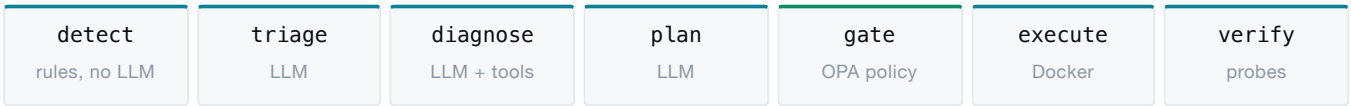
Most self-healing demos sell you the moment it fixes itself. This one sells the receipt: what it saw, what it decided, who approved it, and whether the fault was actually gone afterwards.

WHAT IT IS

AEGIS watches a small but real three-service shop stack. When something breaks, a plain rules engine notices, LLM agents read the metrics, logs and traces, and the system picks a fix from a closed list of actions it is allowed to take. Low-risk fixes run on their own. Risky ones wait for a signed human decision. Every step is written to a hash-chained log you can replay afterwards, frame by frame.

You can run the whole thing offline with no API key. `MOCK_LLM=1` replays recorded model responses, which is how CI runs it and how the demo was recorded.

THE LOOP



Green tier executes immediately. Yellow opens a veto window. Red blocks until a human signs an approval with an Ed25519 key held in their browser, never on the server.

A RUN, MEASURED

7s

to heal a stopped payments service, 13 seconds after detection fired

155

unit and policy tests: 93 Python, 53 TypeScript, 9 OPA

18/19

end-to-end scenarios pass on fixtures. The one that fails is meant to

EARLIER LIVE NUMBERS

Three live runs per scenario against the real API, on a model set this project no longer runs (`gpt-oss-120b` , `gpt-oss-20b` , `qwen3.6-27b`). Kept because they were measured, labelled because they are not current.

SCENARIO	MTTR	COST	ACTION TAKEN
latency	92s	\$0.0025	remove_toxic (green)
crash	61s	\$0.0015	restart_service (green)
error spike	132s	\$0.0030	rollback_config (yellow)
memory leak	21s	\$0.0019	restart_service (green)
cache outage	136s	\$0.0023	remove_toxic (green)

THE TWO MINUTE RECORDING

The demo in the repository is one continuous run with two halves, and the second half is the interesting one. First a service is stopped: detection fires, the agent reads metrics, logs and container stats, policy allows a green tier restart, and it is healed in seven seconds.

Then latency is injected, and the system gets it wrong. It restarts a service that was never down, the fault survives, and the incident closes anyway. The recording ends on the name of the failing test rather than on a success frame, because a demo that only shows the good half is the thing this project is arguing against.

One scenario does not heal, and the test for it stays red

Inject 1500ms of latency and the model proposes restarting a service that was never down. The restart does nothing to a network proxy fault. The incident then closes anyway, because p95 latency is never compared against its threshold at all.

The cause is two words that do not match. The rules file keys the query `p95_latency`. The rule that uses it has the id `latency_p95`. The verification code builds its threshold table under rule ids and then looks it up by query key, so the lookup returns nothing and the comparison is skipped. Error rate escaped this only because its two names happen to be the same string. Latency has gone unchecked since the second week of the project.

`test_latency_heals` fails in CI on purpose. Marking it skipped would make the dashboard green and the system no better.

DIAGNOSIS QUALITY IS THE WEAK POINT

The prompts used to list every injected fault by mechanism, and name the fix with the parameter already filled in. A model handed a symptom could match the list and answer without reading anything. Both lists were deleted. Without them, over five live diagnoses, one model call read the metrics and logs before answering and four answered immediately with no tool call at all.

Across 29 live diagnoses the trace-reading tool has been called zero times. The tool-free answers used to name a fault confidently at 0.8. They now restate the symptom at 0.0. The model was not diagnosing before and it is not diagnosing now. It has only stopped sounding like it was.

WHAT IT DOES PROVE

- **Nothing improvised.** Agents pick a key from a fixed, typed catalog. There is no path from a model response to a shell command, and the executor never imports LLM code.
- **Every action was gated.** Policy evaluates tier, confidence and severity before anything runs, including the actions that turned out to be useless.
- **The log cannot be quietly edited.** Each event hashes the one before it, written in the same transaction as the row it describes. An endpoint recomputes the chain on demand.
- **It admits the miss.** Verification separately re-checks whether the injected fault is still in place, so incidents that closed over a live fault say `[injected fault still present at verify]` in their summary and in the log.
- **There is an export.** One request returns a zip with a timeline PDF, every policy decision, every signed approval, chain verification and cost, plus the raw events.

WHAT IT IS NOT

A production incident response system. It is one Docker Compose host, five chaos scenarios and a free tier API key whose daily quota this project used up more than once. Kubernetes, Temporal, hardware-backed approvals and a bigger action catalog are written down as issues, not built.

Run it yourself: clone, `cp .env.example .env`, `make up`. Docker and Python 3.12 are the only prerequisites, and the default configuration needs no API key. Then open the chaos panel and break something.